AgentDiet

核心：让 agent 把“已经看过、但之后还会反复占上下文”的历史轨迹持续瘦身。

Background

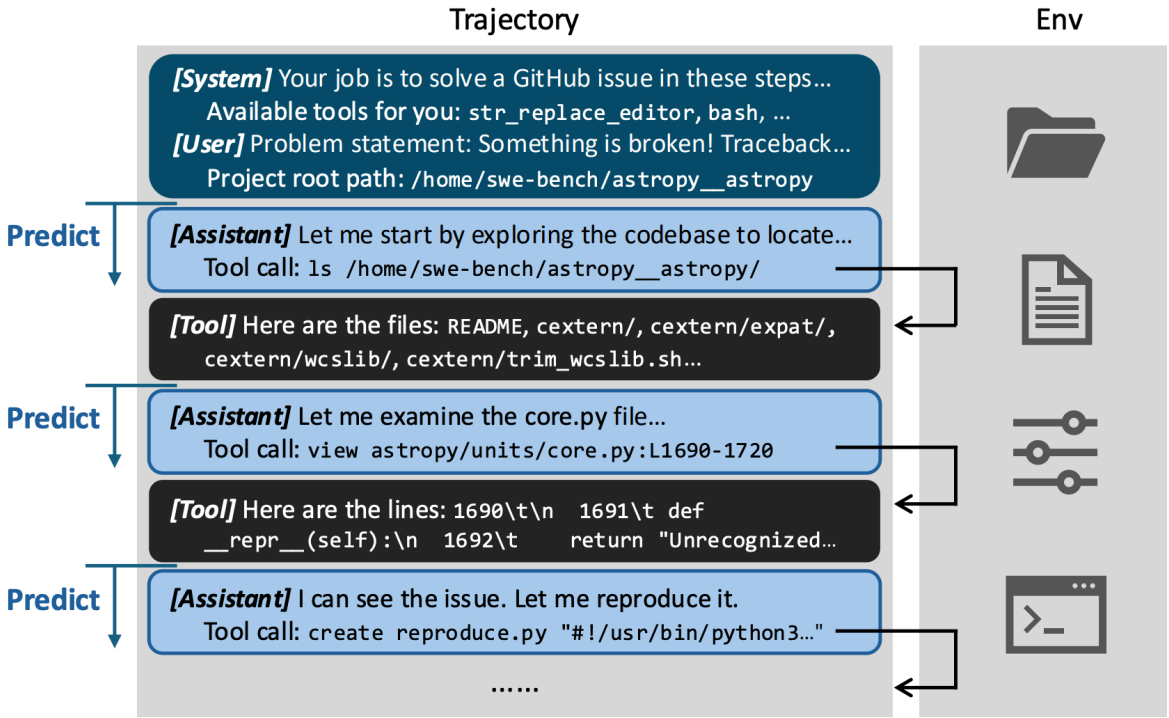

Fig. 1. Typical workflow of an LLM agent.

图 1 画出了一个典型 coding agent 的循环：系统提示和任务描述先进入轨迹，随后模型做一步推理、调用工具、工具返回结果，然后这些 assistant/tool 消息被**永久拼回轨迹**，供下一步继续输入模型。

问题就在这里：哪怕某一步只是打开了一个大文件、跑了一次很长的测试日志，这些长输出也会在后面的每一步里继续重复付费。

作者统计到，在 SWE-bench Verified 上，一个 issue 的平均轨迹大约有 48.4K tokens、40 步，但累计输入开销会膨胀到 1.0M tokens；而且即便有 KV Cache，长轨迹仍然会消耗算力、显存和带宽，改动旧 token 还会让后续 cache 失效。

Insight

作者手工看了 100 条 Trae Agent 的真实轨迹，把可压缩的信息分成三类：

第一类是 **useless information**，也就是对当前任务没什么帮助的东西，比如仓库里缓存目录、资源文件、构建日志里的大量噪音、测试输出里一长串通过用例。

第二类是 **redundant information**，即重复出现的信息，最典型是 `str_replace_editor`：编辑命令里已经给了旧字符串和新字符串，工具返回里又把替换结果重复了一遍，而这些内容很多在更早查看文件时还出现过。

第三类是 **expired information**，即某一步局部有用、之后却过期了的信息，比如 agent 用 `grep` 找多个候选文件，最后真正定位了 bug 文件后，那些其他候选文件的内容通常就没必要继续一直保留。

Approach

用一个单独的 LLM 反思模块（reflection module），在运行时定期压缩旧轨迹中的某一步。

> 注意，这里作者专门强调了一点：他们试过让主 agent 自己调用一个 `erase` 工具去删历史，但即使提示得很明确，强模型也常常继续执着于原任务，不愿进入“反思删减模式”。所以论文最后采用的是**外部控制**：主 agent 继续正常做任务，外层系统在合适时机调用另一个模型，专门负责“给轨迹减肥”。这样做的好处是，压缩时机由系统控制，主 agent 对压缩过程几乎无感，不会被打乱工作流。

> 这个反思模块并不是“把整段历史摘要一下”那么粗糙，而是**只改某一个 step**，而且改法很克制。

作者把一个 step 序列化成 XML 风格的结构，比如 `<step id="19"> ... </step>`，然后给反思模型一个专门设计的 prompt。这个 prompt 有四部分：

1. 先告诉模型它的高层任务：分析并压缩 bug-fixing agent 的某个 step；
2. 再规定输入输出格式：每一步都包在 `<step>` 标签里，输出时尽量保持结构不变；
3. 接着给出前面总结的三类 waste 的例子；

4. 最后给一组“防止删过头”的指导原则，比如删掉内容时最好替换成一句短 takeaway，而不是直接挖空。

   也就是说，它不是自由摘要，而是**结构化局部改写**：保留格式、保留关键信息、只把冗余/过期/无用部分替换成短说明。

压缩例子：

Fig. 2. A case study of reducing information waste in the trajectory (pytest-dev__pytest-6202).

> 例子里 agent 跑了一次 `pytest`，返回了一整页测试输出。原始 step 有 **1995 tokens**，其中大量是“很多测试都 PASSED”的冗长列表。AgentDiet 并没有把整段变成一句模糊摘要，而是把中间那大段重复的通过用例替换成类似 **“individual test lines omitted; mostly PASSED”** 的占位总结，同时保留真正重要的失败信息。于是这一步被压到 **259 tokens**。你可以把它理解成：**不是丢失信息，而是把低价值细节压缩成面向后续决策的简短记忆。**

机制：

设当前 agent 执行到第 (s) 步，AgentDiet 不压最新一步，而是去压**更早的第 (s-a) 步**；压缩时只给反思模型看一个局部窗口：从第 (s-a-b) 步到第 (s) 步。这里：

- a 是“延迟几步再压”，避免刚产生的新信息立刻被动刀；
- b 是给多少前文作为压缩上下文；
- θ 是长度阈值，太短的 step 不值得压；
- 还有一个“收益阈值”判断：如果压缩后省下的 token 不够多，也不应用。

这个设计同时解决了三个问题：

第一，**控制额外开销**，因为反思模型只看一个小窗口，不看全历史；

第二，**减少 KV Cache 失效的代价**，因为只改一个相对较新的旧 step，不会把更早的大段缓存全废掉；

第三，**保护主 agent 的稳定性**，因为最近一步和全部历史都不会被一口气删掉，哪怕反思模型偶尔失误，伤害也被限制住了。图 3 和算法 1 讲的就是这个机制。

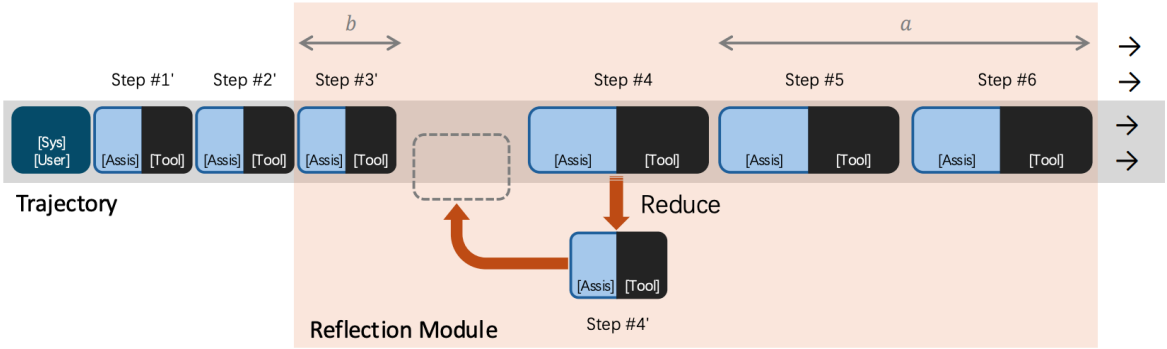

Fig. 3. Design of the reflection module in AgentDiet.

Algorithm 1 Integrating AgentDiet in a typical LLM agent

Input: Problem instruction (I), Environment (E)
Output: Result (r), Modified environment (E)
Constant: Number of steps before (b) and after (a) the target step in context, Step limit ($s_{\max}$), Length threshold (θ), LLM for agent steps ($\text{LLM}_{\text{agent}}$) and reflection steps ($\text{LLM}_{\text{reflect}}$)

$T \leftarrow \textsc{MakeInitialPrompt}(I)$ ▹ Initialize the trajectory
for each $s \in [1 \ldots s_{\max}]$ **do**
  $m_{\text{assis}} \leftarrow \text{LLM}_{\text{agent}}(T)$ ▹ Perform agent step
  if $\textsc{IsTaskDone}(m_{\text{assis}})$ **then**
    $r \leftarrow$ "finished"
    return r, E
  end if
  $E, m_{\text{tool}} \leftarrow \textsc{ExecTool}(E, m_{\text{assis}})$ ▹ Parse and execute the tool call in the environment
  $T \leftarrow T + [\langle m_{\text{assis}}, m_{\text{tool}} \rangle]$ ▹ Concatenate messages in this step into the trajectory

  if $s - a > 0$ **then**
    $l_{\text{orig}} \leftarrow \textsc{Length}(\textsc{Serialize}([T[s-a]]))$
    if $l_{\text{orig}} > \theta$ **then**
      $ctx \leftarrow \textsc{Serialize}(T[\max(0, s-a-b) : s])$
      $m_{\text{reduced}} \leftarrow \text{LLM}_{\text{reflect}}(ctx, s-a)$ ▹ Perform reflection step
      $l_{\text{reduced}} \leftarrow \textsc{Length}(m_{\text{reduced}})$

      if $l_{\text{orig}} - l_{\text{reduced}} > \theta$ **then** ▹ Apply reduction if benefit reaches the threshold
        $T[s-a] \leftarrow m_{\text{reduced}}$
      end if
    end if
  end if

end for

$r \leftarrow$ "interrupted" ▹ Reaching the step limit
return r, E

把算法按执行顺序讲，就是这样：

主 agent 先像平常一样跑：初始化轨迹 (T)，让 `LLMagent(T)` 生成下一步动作，执行工具，把 assistant/tool 消息拼回 (T)。然后 AgentDiet 在每一轮循环尾部检查：如果当前已经至少过去了 (a) 步，就拿第 (s-a) 步做候选；如果这一步长度大于阈值 (θ)，就把窗口上下文序列化后送给 `LLMreflect` 也就是说，**AgentDiet 不是替代 agent，而是插在 agent loop 里的一个条件触发后处理器**。

另一个关键点是：**反思模块用的模型可以比主模型便宜很多。**

作者认为“压缩轨迹”比“真正修 bug”容易，所以没必要用和主 agent 同档次的贵模型。他们比较了 Claude 3.5 Haiku、Gemini 2.5 Flash、GPT-5 mini、DeepSeek v3、Qwen 3 等作为 `LLMreflect`，最后选了 **GPT-5 mini**；再配合超参数搜索，最终设置为 **(** $a = 2,\ b = 1,\ \theta = 500$**)**。这个组合的直觉是：只压“至少有点长”的旧 step，而且等它老两步再压，这样既能省钱，又不太容易误伤后续推理。

Result

在两套 benchmark、两种主模型上，AgentDiet 把处理到的 step 中 **69.2%–77.4%** 的内容删掉，累计输入 token 下降 **39.9%–59.7%**，算上反思模块本身的额外成本后，最终总成本仍下降 **21.1%–35.9%**；而成功率基本与原始 agent 持平，甚至在长上下文更容易“发疯”的设置里还提升了稳健性。图 4 展示的是大量 step 在压缩前后 token 数的直方图，图 5 则展示了某个设置下达到 step limit 的轨迹明显减少。